

JOIN METHOD:

22 February 2025 12:09

- If a thread wants to join another thread to complete its task then we should use join().
- The join() method in Java is used to **pause the execution of the current thread until another thread completes its execution**. It is mainly used to **synchronize multiple threads** and ensure they complete in a specific order.

Why join() is Needed?

Imagine you are performing **multiple tasks** (threads), but one **depends on the result of another**. If we don't use join(), all threads may run **independently**, and the dependent task might execute **before** the required data is ready.

- join() ensures that a thread waits for another thread to finish before proceeding.

Real-World Example: Online Shopping 🛒

Scenario: Placing an Order

Imagine an **online shopping** system where:

1. **Step 1:** The customer **selects** items and places the order.
2. **Step 2:** The system **processes the payment**.
3. **Step 3:** After payment is successful, the system **confirms the order**.

- **Payment must be completed before order confirmation!**

Where is join() Used in Real Life?

1. **Flight Booking System** ✈️
 - The ticket is **confirmed only after payment is completed**.
2. **Bank Transactions** 🏦
 - Funds are **deducted before transaction completion**.
3. **Software Deployment** 💻
 - Code **compilation must finish** before deploying an application.
4. **Gaming** 🎮
 - Player **loading screen must complete** before starting the game.

JOIN():

1. public final void join() throws InterruptedException
2. public final void join(long millis) throws InterruptedException
3. public final void join(long millis, int nanos) throws InterruptedException

CODE:

```
package multithreading;
class Payment extends Thread {
    public void run() {
        System.out.println("Processing payment...");
        try {
            Thread.sleep(3000); // Simulating payment processing time
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println("Payment completed.");
    }
}

class OrderConfirmation extends Thread {
    public void run() {
        System.out.println("Confirming order...");
        System.out.println("Order confirmed! ✅");
    }
}

public class ShoppingExample {
    public static void main(String[] args) {
        Payment payment = new Payment();
        OrderConfirmation confirmation = new OrderConfirmation();
    }
}
```

```

        payment.start();
        try {
            payment.join(); // Order confirmation will wait until payment is completed
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        confirmation.start();
    }
}

```

Key Takeaways

- `join()` **makes one thread wait** until another thread completes execution.
- It is used for **synchronizing dependent tasks** in multithreading.
- **Best suited** for order-based executions like payments, transactions, and data processing.

Interview Questions Related to `join()` Methods

1. What are the different `join()` methods in Java?
2. How is `join(long millis)` different from `join()`?
3. What happens if we call `join()` inside the `run()` method of the same thread?
4. Can a thread be interrupted while waiting in `join()`?
5. What happens if `join()` is called on a thread that has already finished execution?
6. How does `join(long millis, int nanos)` work, and when should we use it?

DIFFERENCE B/W `sleep()`, `yield()` and `join()`

Feature	<code>sleep()</code>	<code>yield()</code>	<code>join()</code>
Purpose	Makes the current thread sleep (pause execution) for a specified time.	Suggests to the scheduler that the current thread is willing to yield CPU to other threads of equal or higher priority.	Waits for another thread to finish before continuing execution.
Defined In	<code>Thread</code> class	<code>Thread</code> class	<code>Thread</code> class
Type of Method	<code>static</code> method	<code>static</code> method	Instance method
Behavior	Pauses execution for a fixed time but does not release locks.	May or may not give control to another thread (depends on scheduler).	Forces the current thread to wait until the specified thread completes.
Time Specification	Requires a time duration (in milliseconds).	No time duration; only a request to yield CPU.	Can be called with or without a timeout.

Effect on CPU	Blocks the current thread for a given time, reducing CPU usage.	Does not block; only hints to the scheduler to switch threads.	Does not release CPU but ensures thread completion before proceeding.
Lock Impact	Does not release acquired locks.	Does not release locks.	Does not release locks.
Throws Exception	<code>InterruptedException</code>	No exception	<code>InterruptedException</code>
Use Case	Used when you want to delay thread execution.	Used when you want to let other threads run but without forcing a wait.	Used when you need a thread to complete before moving forward.